ACIT 4850 – Enterprise Systems Integration – Assignment 2

Instructor	Mike Mulder (mmulder10@bcit.ca or Discord) Johnny Zhang (johnny_zhang@bcit.ca)
Total Marks	24
Due Dates	March 23rd at midnight

This assignment is to be completed and submitted with your partner (i.e., Group submission) if applicable.

Overview

This time you already have a solution for the tool you are evaluating in your Enterprise Development Environment: Jenkins for CI/CD. But your software development team is looking at alternative new options – because developers like trying out new tools – for Continuous Integration.

Jenkins vs. GitLab CI

We already have Jenkins in our Enterprise Development Environment as of Lab 4 for Continuous Integration. And in Lab 8 you prototyped the use of GitLab CI for our Point Python Pipeline using a Shell Runner. Now you are going to prototype GitLab CI for our Java Sample Pipeline using a Docker Runner.

Part A – Practical (14 marks)

- You must configure GitLab CI on your Cloud VM (you cannot use the online version of GitLab CI).
- Reproduce your Java Sample Pipeline using GitLab CI with a **Docker Runner**. You can use the Maven docker image which has Java and Maven both installed by default (https://hub.docker.com/_/maven)
- The stages you must include are:
  - Build
  - Check (New)
  - Test
  - Deliver

  Note: You do not have to reproduce the SonarQube integration (i.e., you can modify the Build stage to remove the SonarQube specific syntax and remove the Quality Gate stage altogether).
- The Build stage builds the Java JAR file that is then run in the Deliver stage. However, in GitLab CI build outputs aren't carried over from stage to stage when using a Docker runner. **You must investigate and use caching in GitLab CI to deal with this issue.**

The Check stage is a custom stage that you will need to add to the pipeline:

- Using Maven, run the pmd:pmd target in this stage using the Apache PMD Plugin static code analyzer: https://maven.apache.org/plugins/maven-pmd-plugin/index.html
- Questions:
  - What is the purpose of the PMD Maven Plugin?
  - What PMD code violation do you get?
  - What is the difference between the pmd:pmd and pmd:check targets? Show the output for each in your pipeline.
  - In general, what causes a Stage/Job in a Pipeline to fail (other than an error in the pipeline itself)?

Submission:

- (2 marks) Two screenshots of your Docker runner, one from the GitLab UI and one from the VM, proving that it is a Docker runner that is running on your GitLab VM.
- (2 marks) Screenshot of your pipeline running successfully with the above stages. It must show the URL of your GitLab installation (with your Group #).
- (2 marks) Screenshot of the output of your Deliver stage showing the output of the built Java program. It must show the URL of your GitLab installation (with your Group #).
- (4 marks) The answers to your questions on the Check stage (above).
- (4 marks) Your .gitlab-ci.yml file with the stages above and leveraging the Maven docker image and caching. Make sure to include a comment in your .gitlab-ci.yml file with your Group # and names. Note: You will lose an additional two marks here if the Check stage isn't included.

Part B – Written

The answers to the following questions should reflect your hands-on experience with both Jenkins and GitLab CI and less on online research (though you may have to do some research and experimentation):

Answer the following questions comparing Jenkins and GitLab CI:

1. (Up to 3 marks) Identify three similarities between Jenkins and GitLab CI. For at least two of the similarities, in addition to describing it, you must:
   - Reference the specific lab(s) or pipeline(s) where you noticed it
   - Quote or describe the specific implementation detail or configuration element (e.g., Jenkinsfile syntax, .gitlab-ci.yml structure, agent/runner configuration, stages definition, etc.)
   - Provide one or more screenshots showing the similarity

   Note: Answers that do not reference your own implementation details will not receive full marks.

2. (Up to 3 marks) Identify three differences between Jenkins and GitLab CI. For at least two of the differences, in addition to describing it you must:
   - Reference the specific lab(s) or pipeline(s) where you noticed it
   - Quote or describe the specific implementation detail or configuration element (e.g., Jenkinsfile syntax, .gitlab-ci.yml structure, agent/runner configuration, stages definition, etc.)
   - Provide one or more screenshots showing the difference
   - Explain which tool you found more intuitive or more difficult and why (based on your actual use)

3. (Up to 4 marks) Define two scenarios – i.e., companies with software teams – that would use a CI/CD tool. For one scenario, recommend and justify Jenkins as the preferred choice. For the other scenario, recommend and justify GitLab CI as the preferred choice. *Your justifications must reference specific configuration characteristics from your hands-on experience from the labs to receive full marks.*

Submission

Submit your screenshots, .gitlab-ci.yml file and PDF with your answers to the written portion to the Assignment 2 dropbox by March 23rd at midnight.